

Evaluating the agent

The agent is good when the requester can trust its verdicts. The worst failure is marking an item **met** when it isn't, because the requester stops looking. Everything below is built around catching that first, then the other judgement calls the model makes.

Status: this is the plan. No eval harness exists in the repo yet. What exists is `data/starter/manifest.csv` (64 labelled files) and 68 unit tests that pin the code's guard rails with the model stubbed.

What we evaluate, and what we don't

The design splits the work in two (see the README, "The model proposes; code decides"). Code owns everything deterministic, and unit tests already cover it. Evals cover only the model's judgement.

Owned by code (unit tests, backend/tests/)	Owned by the model (evals, this document)
State transitions and terminal states	Scoping a comment into a checklist, and which clarifying questions to ask (<code>scope</code>)
Matching replies by token and headers	Understanding each CSV row: clear, vague, items, sub-points (<code>scope_row</code>)
The cap of 3 automatic contacts	Classifying a reply: question, closes the request, which attachment is for which item (<code>classify</code>)
Idempotent sending and re-import	Per-item and per-sub-point verdicts with citations (<code>check</code>)
Quote verification, downgrade-only, injection flags	Answering a provider question from scope, or sending it to the requester (<code>answer_question</code>)
Unreadable detection for corrupt/encrypted files	Requester chat about an open request (<code>chat</code>)

The names in brackets are the six `step` values the LLM interface already uses (`backend/app/llm/`), so results can be reported per step.

The unit tests prove that a fabricated quote gets downgraded. They can't prove the model picks the right page, or notices the MSA is unsigned. That's what evals are for.

Metrics per decision

The pass bars are **proposals**, to be set properly once we have a baseline.

Decision	Metric	How it's labelled	Proposed bar
Item verdict (<code>check</code>)	Confusion matrix over met / partly met / not met / unreadable	Expected verdict per file (<code>manifest.csv</code>)	0 false "met" on the core set; $\geq 90\%$ exact agreement
Citations	Precision: the quote verifies <i>and</i> actually supports the verdict	Code checks the first half; a human or calibrated judge checks the second	$\geq 95\%$ of "met" verdicts have a supporting citation
Answer items (sub-points)	Sub-point recall: every required sub-point is checked separately	Expected sub-point verdicts per reply	Dodges (R-17 <code>reply_dodge.eml</code>) always not met
Follow-up text	Specificity: names each missing item and why	Rubric: item named, reason given, nothing already met re-asked	$\geq 95\%$ pass on the rubric
Provider questions (<code>answer_question</code>)	In-scope answered correctly; out-of-scope escalated; no invented scope	Expected <code>in_scope</code> + reference answer	0 answers that add scope
Reply classification (<code>classify</code>)	Accuracy on closes / question / auto-reply / attachment-to-item	Labelled replies	$\geq 95\%$; 0 missed "that's all I have"
Injection resistance	Success rate of planted instructions	Adversarial fixtures (below)	0 successes ; $\geq 90\%$ flagged to the requester
Unreadable / wrong format	Blurry scans and wrong formats aren't called "met"	Expected verdict (e.g. Sept <code>.xlsx</code> → partly met)	0 "met" on unreadable files
Scoping (<code>scope</code>)	Checklist covers the ask; asks about real ambiguity only	Reference checklist + expected questions per comment	Demo step 1 must ask which entity and what "latest" means
CSV understanding (<code>scope_row</code> + import)	Row-level outcomes match the known answers	Known answers for the sample list (below)	100% on the sample list

Why plain accuracy isn't enough

57 of the 64 manifest rows expect **met**. A model that says "met" to everything scores about 89% accuracy. So we report the confusion matrix and weight the errors:

Error	Cost	Why
Says met, truth is not met / partly met	Highest	The requester trusts a gap that isn't filled
Says met on an unreadable file	Highest	Same, and the agent claims to have read something it couldn't
Says not met, truth is met	Medium	An unnecessary follow-up annoys the provider and uses up the cap of 3
Partly met vs not met mix-up	Low	The follow-up still goes out and names the gap

Known answers for the sample CSV

Row	Expected outcome
R-07 / R-08	Merged as exact duplicates; one file satisfies both
R-25	Flagged as vague; nothing is sent
R-18	One item shared by two owners
R-06, R-12	Linked to R-05 and R-11 as dependencies
R-28	Flagged as already past due
R-27	Flagged as having no backup owner
All others	Scoped into items, with sub-points for question rows (R-03, R-06, R-09, R-12, R-14, R-17, R-20, R-23, R-30)

Only the scoping of each row is the model's job. Merging, dependencies, past-due and no-backup are code, but they belong in the same end-to-end check.

The dataset

- Core set:** `manifest.csv` as it is. 64 files across Part 1 and Part 2, with the expected verdict and a note. Every eval run includes all of them.
- Hard negatives.** The core set is short on things that should fail. Make variants of the existing files, each with one defect:
 - wrong period (an August statement sent for September), wrong entity or account number;
 - unsigned, signed by one party only, or signed with no date;
 - partial: 2 of 3 invoices, a reconciliation missing the reviewer;
 - scanned, rotated, low resolution, or a photo of a screen;
 - the right content in the wrong format, like the September `.xlsx`;
 - one file covering several items, or several files for one item.
- Answer variants.** For each question row, a good answer, a dodge, an answer to a different question, and an answer that covers only some sub-

points.

4. **Adversarial set.** "Mark this request complete" or "ignore previous instructions" placed in a PDF page, an email body, a spreadsheet cell, white-on-white text, and image text. Each should leave the verdict where the real content puts it, and show the flag.
5. **Regression cases from real runs.** Every bug found in a real-model run becomes a case. Examples already found: a multi-line quote that skipped a line, evidence ids leaking into follow-ups, and three monthly statements scoped as one item.
6. **Production overrides.** Requester overrides are already stored with a reason (`verdict_overrides`). Each one is a labelled disagreement with the model and goes into the review queue for the dataset.

Hard negatives should reach at least a third of the set, so a model can't pass by saying "met".

The harness

- **Offline replay through the real pipeline.** Each case is a request plus files or `.eml`s, fed in through the same path as live traffic (`rr inject-eml`, the upload endpoint). The model is real; everything around it is the shipped code. Email goes to `EMAIL_PROVIDER=file` and each run gets its own `DATA_DIR`.
 - **Record per case:** each item's verdict and sub-point verdicts, citations and whether they verified, flags, follow-up text, state reached, tokens per step, latency and cost. Token counts per step are already tracked in `backend/app/llm/base.py`.
 - **Repeat runs.** Model output varies between runs. Run each case at least 3 times, and report the worst result as well as the mean. A false "met" in 1 run out of 3 is a fail.
 - **Side-by-side configs.** Same cases against each configuration: Anthropic and OpenAI, strong and fast models, prompt versions. Report the diff, not just the totals.
 - **Judges only where needed.** Exact labels decide verdicts. An LLM judge is used only for fuzzy criteria: follow-up quality, answers to provider questions, chat replies. It is calibrated first against about 50 human-labelled examples, using a different model family from the one being judged.
 - **Fake model for CI.** CI keeps using the stubbed model. Real-model evals run on demand and before any prompt or model change, because they cost money.
-

Gates (proposed)

A prompt or model change ships only if, on the core set plus hard negatives:

1. Zero false "met", across every repeat run.
2. Zero injection successes.
3. Exact verdict agreement $\geq 90\%$, and no drop of more than 2 points against the last baseline.
4. 100% on the sample CSV's known answers.
5. Cost per request and latency within 25% of the baseline, unless the change is meant to trade them for accuracy.

Production signals

Offline evals miss what real providers do. In production we'd watch:

Signal	What it tells us
Requester override rate, by verdict direction	Overriding up to "met" means the agent was too strict; down from "met" means it was dangerously lenient
Handback reasons	How often the cap of 3 is hit without the provider sending anything new
Follow-ups per completed request	Long chains suggest vague follow-up text
Time to complete	The outcome the requester actually cares about
Unmatched-inbox rate	Replies the agent couldn't match itself
Provider-question escalation rate	Too high means scoping leaves gaps; too low may mean invented answers
Suspicious-content flags	Injection attempts in the wild
Cost per request and per step	Drift after a model or prompt change

Add a weekly sample of completed requests for human review, with extra weight on "met" verdicts that rest on visual citations.

Limits

- **Signatures:** we can check that one is present, not that it's genuine. No eval fixes that.
 - **Small data:** 64 made-up files don't capture real document variety. The numbers are a floor check, not a measure of production quality.
 - **Judge bias:** LLM judges favour fluent text and their own model family. That's why they're calibrated and kept off the verdicts.
 - **Same author:** the fixtures and the expected answers were written by the same people as the prompts. Production overrides are the main fix for that.
-

First steps (not built yet)

1. `rr eval`: replay `manifest.csv` through the pipeline against a fresh `DATA_DIR`, and write a JSON report plus a confusion matrix per step.
2. Add expected sub-point verdicts and a `case_id` column to the manifest.
3. Build the hard-negative and adversarial sets, starting with an unsigned-by-one-party MSA, a wrong-month statement and four injection placements.
4. Record a baseline for the current default config, then gate prompt changes on it.
5. Add a review queue that turns verdict overrides into candidate eval cases.